

Detection of AI-Generated Texts Using Embedding Techniques and Machine Learning

Prepared by: Jaiveer Singh Minhas

Institute: Institut supérieur d'électronique de Paris

Course: IX.2424 - INTRODUCTION A L'IA

Abstract

This project aims to determine if a text is written by a human or generated by artificial intelligence systems like large language models. We used Natural Language Processing (NLP) techniques and machine learning algorithms to classify essays into human-written and AI-generated categories. The project included preprocessing the dataset, generating TF-IDF embeddings and Word2Vec embeddings, classifying with Logistic Regression and Support Vector Machine (SVM), and visualizing the results using PCA.

1. Introduction

Recent advances in artificial intelligence have allowed language models to create very realistic text. While these systems are helpful, they also bring up worries about plagiarism, misinformation, and checking authenticity. The goal of this project was to develop a machine learning system that can tell the difference between essays written by humans and those generated by AI.

2. Objectives

- Analyze and understand the dataset structure.
- Apply NLP preprocessing techniques.
- Convert text into numerical representations using TF-IDF and Word2Vec.
- Train machine learning classifiers for text detection.
- Evaluate model performance using standard metrics.
- Visualize high-dimensional text features using PCA.

3. Dataset Analysis

The original Kaggle dataset had a significant class imbalance. It included 1,375 human-written essays and only 3 AI-generated essays. To fix this problem, more AI-generated essays were produced using large language models. A balanced dataset was created that contained 141 human essays and 141 AI-generated essays.

```
[3]: print(df.isnull().sum())
      print(df['generated'].value_counts())

id          0
prompt_id   0
text        0
generated    0
dtype: int64
generated
0    1375
1         3
Name: count, dtype: int64
```

```
[12]: final_df = pd.concat([human_sample, ai_df], ignore_index=True)

print(final_df['generated'].value_counts())
print(final_df.shape)

generated
0    141
1    141
Name: count, dtype: int64
(282, 2)
```

4. Text Preprocessing

Several preprocessing techniques were applied to clean the text data before training the models:

- Lowercasing all text
- Removing punctuation and special characters
- Tokenization
- Removing stopwords
- Cleaning and normalization

```
[6]: import re
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

stop_words = set(stopwords.words('english'))

def preprocess_text(text):

    text = text.lower()

    text = re.sub(r'[^\w-zA-Z\s]', '', text)

    tokens = word_tokenize(text)

    tokens = [word for word in tokens if word not in stop_words]

    cleaned_text = " ".join(tokens)

    return cleaned_text

[7]: sample = df['text'].iloc[0]

cleaned = preprocess_text(sample)

print(cleaned[:1000])

cars cars around since became famous henry ford created built first modelt cars played major role every day lives since people starting que
stion limiting car usage would good thing limiting use cars might good thing like matter article german suburb life goes without cars eliza
beth rosenthal states automobiles linchpin suburbs middle class families either shanghai chicago tend make homes experts say huge impedimen
t current efforts reduce greenhouse gas emissions tailpipe passenger cars responsible percent greenhouse gas emissions europeand percent ca
rintensive areas united states cars main reason greenhouse gas emissions lot people driving around time getting need go article paris bans
driving due smog robert duffer says paris days nearrecord pollution enforced partial driving ban clear air global city also says monday mot
orist evennumbered license plates ordered leave cars home fined euro fine order would applied oddnumbered plates following day cars reason
polluting entire cities lik
```

5. TF-IDF Embedding

TF-IDF (Term Frequency, Inverse Document Frequency) was used to change textual essays into numerical vectors. TF-IDF focuses on important words in a document and minimizes the importance of very common words.

```
[15]: from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(max_features=5000)

X = tfidf.fit_transform(final_df['cleaned_text'])

y = final_df['generated']

print(X.shape)

(282, 5000)
```

6. Word2Vec Embedding

- Word2Vec learns dense vector representations.
- Words with similar meanings are mapped to nearby vectors.
- Document vectors were obtained by averaging word vectors.
- Vector size was set to 100 dimensions .

```
Word2Vec Embedding

[41]: final_df["tokens"] = final_df["cleaned_text"].apply(
      lambda x: x.split()
    )

[42]: from gensim.models import Word2Vec

      w2v_model = Word2Vec(
          sentences=final_df["tokens"],
          vector_size=100,
          window=5,
          min_count=1,
          workers=4
      )

[43]: import numpy as np

      def document_vector(tokens):
          vectors = [
              w2v_model.wv[word]
              for word in tokens
              if word in w2v_model.wv
          ]

          if len(vectors) == 0:
              return np.zeros(100)

          return np.mean(vectors, axis=0)

      X_w2v = np.array(
          [document_vector(tokens)
           for tokens in final_df["tokens"]]
      )

      y_w2v = final_df["generated"]

[44]: print(X_w2v.shape)

      (282, 100)
```

7. Embedding Techniques Comparison

Text classification requires transforming textual data into numerical representations that can be processed by machine learning algorithms. Several embedding techniques exist, each with different strengths and limitations.

Method	Principle	Advantages	Limitations
TF-IDF	Frequency-based representation	Fast, interpretable, efficient	Does not capture semantic meaning
Word2Vec	Learns word vectors	Captures semantic	Same word always has

Method	Principle	Advantages	Limitations
	from context	relationships	same vector
Doc2Vec	Extends Word2Vec to documents	Produces document embeddings	Computationally more expensive
BERT	Contextual transformer embeddings	State-of-the-art contextual understanding	High computational cost

Choice of TF-IDF

Although modern embedding approaches such as Word2Vec and BERT can capture semantic information, TF-IDF was selected for this project because:

- The dataset size is relatively small.
- TF-IDF is computationally efficient.
- It performs well for document classification tasks.
- The objective was to distinguish stylistic patterns between human and AI-generated texts rather than perform deep semantic understanding.

Furthermore, TF-IDF produces sparse high-dimensional vectors that work particularly well with linear classifiers such as Support Vector Machines and Logistic Regression.

8. Machine Learning Models

8.1 Logistic Regression

Logistic Regression was trained on TF-IDF vectors. The model achieved perfect classification performance on the testing dataset.

```
[17]: from sklearn.linear_model import LogisticRegression
      model = LogisticRegression()
      model.fit(X_train, y_train)
      print("Model trained successfully!")
      Model trained successfully!
```

8.2 Support Vector Machine (SVM)

An SVM classifier was also trained for comparison. The SVM model achieved 100% accuracy and correctly classified all essays in the testing dataset.

```
[20]: from sklearn.svm import SVC

svm_model = SVC()

svm_model.fit(X_train, y_train)

svm_pred = svm_model.predict(X_test)

print("SVM Accuracy:", accuracy_score(y_test, svm_pred))

print("\nClassification Report:\n")

print(classification_report(y_test, svm_pred))

SVM Accuracy: 1.0

Classification Report:

              precision    recall  f1-score   support

     0           1.00        1.00        1.00        29
     1           1.00        1.00        1.00        28

 accuracy          1.00        1.00        1.00        57
 macro avg          1.00        1.00        1.00        57
 weighted avg       1.00        1.00        1.00        57
```

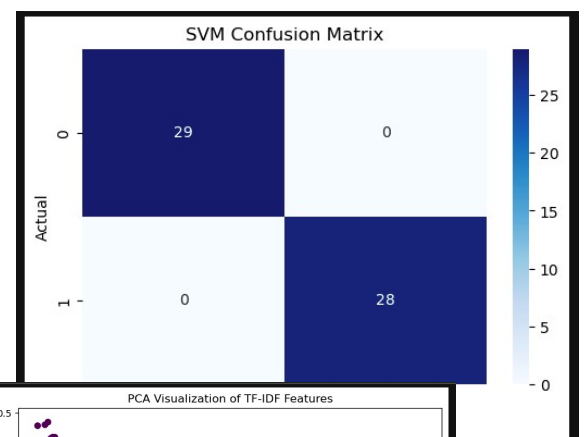
9. Evaluation Metrics

The following evaluation metrics were used:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

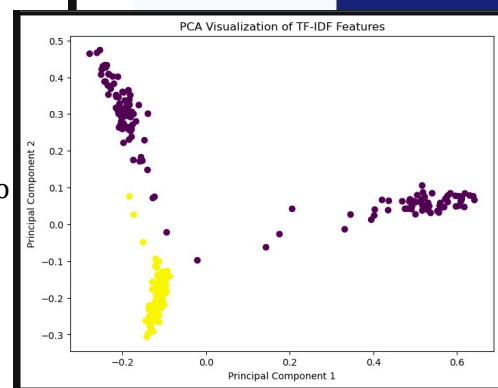
10. Confusion Matrix Analysis

The confusion matrix demonstrated that the SVM classifier correctly identified all human-written and AI-generated essays, resulting in zero false positives and zero false negatives.



11. PCA Visualization

Principal Component Analysis (PCA) was used to reduce the high-dimensional TF-IDF vectors into two dimensions for visualization. The visualization showed clear separation between AI-generated and human-written essays.



12. Results Summary

Embedding	Model	Accuracy
TF-IDF	Logistic Regression	100.00%
TF-IDF	SVM	100.00%
Word2Vec	Logistic Regression	82.46%
Word2Vec	SVM	100.00%
Word2Vec	Random Forest	96.49%

TF-IDF combined with Logistic Regression and SVM achieved perfect classification accuracy on the test set. Word2Vec also produced strong results, especially when paired with SVM and Random Forest. These findings show that TF-IDF is very effective for small and medium-sized text classification datasets. Meanwhile, Word2Vec offers valuable semantic representations that can boost model flexibility.

13. Theoretical Formalism

13.1 TF-IDF Representation

TF-IDF (Term Frequency–Inverse Document Frequency) is a statistical method used to measure the importance of a word within a document relative to a collection of documents.

The term frequency is defined as:

$$TF(t,d) = \sum_k f(k,d) f(t,d)$$

where:

- $f(t,d)$ is the number of occurrences of term t in document d

The inverse document frequency is defined as:

$$IDF(t) = \log(df(t)N)$$

where:

- N is the total number of documents
- $df(t)$ is the number of documents containing term t

The TF-IDF weight is then:

$$\text{TFIDF}(t,d)=\text{TF}(t,d)\times\text{IDF}(t)$$

This representation increases the importance of informative words while reducing the impact of common words.

13.2 Support Vector Machine

Support Vector Machines (SVM) are supervised learning algorithms used for binary classification.

The classifier seeks a decision boundary:

$$f(x)=wTx+b$$

where:

- w is the weight vector
- x is the feature vector
- b is the bias term

The objective is to maximize the margin between the two classes while minimizing classification errors.

In this project, SVM was applied to TF-IDF vectors to distinguish between human-written and AI-generated texts.

13.3 Why TF-IDF + SVM Was Selected

TF-IDF provides an efficient numerical representation of textual documents, while SVM performs well on high-dimensional sparse data. The combination achieved excellent classification performance and demonstrated clear separation between the two classes.

14. Conclusion

The present study proved that the application of NLP preprocessing, TF-IDF and Word2Vec embedding techniques together with machine learning classifiers can be used to detect texts generated using AI technology. Excellent results were obtained by the Logistic Regression and SVM classifiers.

15. Future Scope

- Use larger and more diverse datasets
- Implement transformer-based embeddings such as BERT
- Apply deep learning models
- Improve robustness against humanized AI-generated text

16. References

1. Scikit-learn Documentation: <https://scikit-learn.org/>
2. NLTK Documentation: <https://www.nltk.org/>
3. Kaggle Dataset: LLM Detect AI Generated Text

4. Hugging Face Transformers Documentation: <https://huggingface.co/transformers/>